

ALC Item Sheet Operating Manual

Quick Navigation

- Purpose and operating principle
- Clean baseline and go-live start
- Official files and what operators edit
- Daily, invoice-day, and month-end workflow
- Due date vs billing date vs month close
- Outputs, controls, and audit trail
- Roles, guardrails, and recovery

Purpose

This manual defines how Acento Leasing Company (ALC) should operate the asset lease tracker in production.

The system is designed to stay simple:

- Operators edit only assets and rates
- The script builds and updates lease schedules
- Daily run monitors the portfolio and upcoming billing
- Invoice-day run posts invoice history and refreshes the workbook
- Month-end run records bank payable and locks the month
- Auditability is preserved through the posted ledger, closed periods, manifests, and backups

Operating Principle

The company should run from the current data and the generated reports, not from memory.

That means:

- Assets and rates are the business inputs
- Posted invoices are historical actuals
- Closed months are locked
- Future projections can change only for open periods
- History begins only after real go-live activity

Clean Baseline and Go-Live

Baseline is a one-time clean production start.

At baseline:

- The system creates baseline metadata
- Posted invoice history starts empty
- Closed periods start empty
- Bank payable history starts empty
- No fake, sample, or training transactions should remain in the live production files

Baseline is not:

- a month close
- an invoice run
- a backfill of prior history
- a training dataset

Recommended baseline command:

- ``ALC/scripts/op_init_baseline.sh german 2026-07-01 "Production start"```

Expected result after baseline:

- ``baseline_config.json`` exists with go-live metadata
- ``posted_invoices.csv`` contains header only
- ``closed_periods.csv`` contains header only
- ``run_manifests/`` contains the baseline manifest
- ``backups/`` contains any pre-write file snapshots created by the baseline run

Official Files

The live ALC folder should use one official file set.

User-Managed Business Inputs

- ``ALC/assets.csv``
- ``ALC/rates.csv``

System / Control Files

- ``ALC/baseline_config.json``
- ``ALC/posted_invoices.csv``
- ``ALC/closed_periods.csv``

Operational Outputs

- ``ALC/invoices_YYYY-MM.csv``
- ``ALC/bank_payable.csv``

- `ALC/ALC - Asset Calculation Unit.xlsx`

Audit / Traceability

- `ALC/run\_manifests/`
- `ALC/backups/`

What Operators Edit

Operators should edit only:

- `assets.csv`
- `rates.csv`

Operators should not manually edit during normal operation:

- `posted\_invoices.csv`
- `closed\_periods.csv`
- `bank\_payable.csv`
- files inside `run\_manifests/`
- files inside `backups/`

Official Operator Command Set

Routine operation should use only these wrapper scripts:

- baseline setup: `ALC/scripts/op\_init\_baseline.sh`
- daily run: `ALC/scripts/op\_daily.sh`
- invoice-day run: `ALC/scripts/op\_invoice.sh`
- month-end run: `ALC/scripts/op\_month\_end.sh`

Examples:

- baseline: `ALC/scripts/op\_init\_baseline.sh german 2026-07-01 "Production start"`
- daily: `ALC/scripts/op\_daily.sh german 2026-07-21 22`
- invoice day: `ALC/scripts/op\_invoice.sh german 2026-07 22`
- month-end: `ALC/scripts/op\_month\_end.sh german 2026-07 22`

The optional last argument is the billing day. If omitted, the default is `22`.

Asset Lifecycle and Schedule Logic

Each asset begins when it is entered in `assets.csv` with its activation / start date.

The script uses that start date to build the asset's full projected lease life.

For each asset, the schedule includes:

- opening row at the asset start date
- one expected installment due date per period
- interest and amortization by period
- remaining balance through maturity

The schedule math uses:

- lease base = asset value + tax + admin expense + risk cost recovery - salvage value
- effective monthly rate derived from annual bank APR + NIM
- payment months = term months - salvage periods

Future open rows can change when rates change. Closed historical rows must not change.

Due Date vs Billing Date vs Month Close

These three concepts must be kept separate.

1. Asset Due Date

Each asset has its own due date pattern based on its activation / start date anniversary.

Examples:

- if an asset starts on the 23rd, future due dates follow the 23rd pattern
- if a month has fewer days, the date is clamped to the last valid day of that month

2. Billing Date

Invoices are posted in one monthly batch using the billing day.

Rules:

- default billing day is `22`
- if the billing day falls on a weekend, the script shifts it to the next working day
- the invoice-day run posts all asset installments whose due dates fall between the previous billing run date and the current billing run date

3. Month Close

Month close is a separate control step.

At month-end the script:

- updates the monthly bank payable summary
- marks the month closed in `closed\_periods.csv`

Once a month is closed, it is treated as locked history.

Daily Workflow

Every day the operator should run the daily script once.

Command:

- ``ALC/scripts/op_daily.sh [billing_day]``

Example:

- ``ALC/scripts/op_daily.sh german 2026-07-21 22``

Daily run does not post history. It monitors the current state.

What the daily run prints:

- portfolio totals
- billing month to watch
- billing date
- invoice lines due in the billing window
- invoice amount due in the billing window
- reminder the day before invoice day
- reminder on invoice day

Use the daily run to catch:

- newly added assets
- rate changes already entered
- assets moving toward maturity
- invoice-day timing without relying on memory

Invoice-Day Workflow

Use one invoice-day run per billing cycle.

Command:

- ``ALC/scripts/op_invoice.sh [billing_day]``

Example:

- ``ALC/scripts/op_invoice.sh german 2026-07 22``

What the invoice-day run does:

- reads current assets and rates
- reads posted invoice history first
- determines the billing date for the selected month

- selects asset installments due between the previous billing date and the current billing date
- creates the invoice CSV for the month
- writes posted rows to `posted\_invoices.csv`
- refreshes `ALC - Asset Calculation Unit.xlsx`
- writes a run manifest and any needed backups

What the invoice-day run does not do:

- it does not close the month by itself
- it does not overwrite closed history

Workbook Refresh on Invoice Day

The invoice-day run automatically refreshes the workbook.

Workbook structure:

- first tab: `Inventory`
- remaining tabs: one tab per asset

Workbook behavior:

- all assets remain represented, whether active or inactive
- the Inventory tab shows current asset status and balances
- each asset tab shows lease inputs, lifecycle status, and projected / actual schedule state

Month-End Workflow

Use one month-end run after invoice-day activities are complete.

Command:

- `ALC/scripts/op\_month\_end.sh [billing\_day]`

Example:

- `ALC/scripts/op\_month\_end.sh german 2026-07 22`

What the month-end run does:

- calculates bank payable for the month
- updates `bank\_payable.csv`
- writes one row per month in that file
- closes the month in `closed\_periods.csv`
- writes manifests and backups

Bank Payable File

`bank\_payable.csv` stores one row per month with:

- month
- billing date
- invoice line count
- bank payable total
- operator
- update timestamp

If the same month-end run is repeated for the same month, the row is updated rather than duplicated.

History and Locking Rules

The system uses two kinds of schedule rows:

- projected future rows
- posted historical rows

Rules:

- posted invoice rows are historical actuals
- future runs must reuse posted rows instead of recalculating them
- closed months must not be changed in normal operations
- if a correction is needed after close, the default rule is to post the correction in a later open month
- only authorized admin use should invoke a closed-period override route

Reports and Outputs

Daily Monitoring Output

Daily run provides terminal monitoring output and a run manifest.

Invoice-Day Outputs

Invoice-day run produces:

- `invoices\_YYYY-MM.csv`
- updated `posted\_invoices.csv`
- refreshed `ALC - Asset Calculation Unit.xlsx`
- run manifest
- backups when files are overwritten

Month-End Outputs

Month-end run produces:

- updated `bank\_payable.csv`

- updated `closed\_periods.csv`
- run manifests
- backups when files are overwritten

Audit Trail and Recovery

The script maintains operational traceability through:

- `baseline\_config.json`
- `posted\_invoices.csv`
- `closed\_periods.csv`
- `bank\_payable.csv`
- `run\_manifests/\*.json`
- `backups//...`

Each manifest should record:

- run ID
- operator
- command and timestamp
- month or as-of date where relevant
- billing day and billing date where relevant
- totals and row counts
- input hashes before and after when applicable
- backup references

Recovery principle:

- if a generated file is corrupted, restore from the backup created before the last write operation
- use manifests to identify the exact run that changed the file

Roles and Guardrails

Operator

- update `assets.csv`
- update `rates.csv`
- run the wrapper scripts
- review daily output and generated files
- verify that invoice-day and month-end outputs were produced

Admin / Owner

- manage go-live baseline

- authorize exceptional closed-period adjustments if ever needed
- manage structural changes to files or workflow

Guardrails

- never edit `posted\_invoices.csv` manually during normal operation
- never edit `closed\_periods.csv` manually during normal operation
- never create fake production history after baseline
- use wrapper scripts for routine operations

Recommended Daily Decision Flow

1. Update `assets.csv` if any asset changed.
2. Update `rates.csv` if the bank published a new rate.
3. Run the daily script.
4. Review totals, billing date, and reminders.
5. If today is invoice day, run the invoice-day script.
6. If month-end processing is due, run the month-end script.

Success Criteria

The operating model is working correctly when:

- operators edit only assets and rates
- daily run is used consistently for monitoring
- invoice-day run posts asset-level billing history by billing window
- month-end run updates bank payable and closes the month
- workbook stays current without separate manual refresh steps
- locked history remains unchanged after close
- manifests and backups exist for every write operation